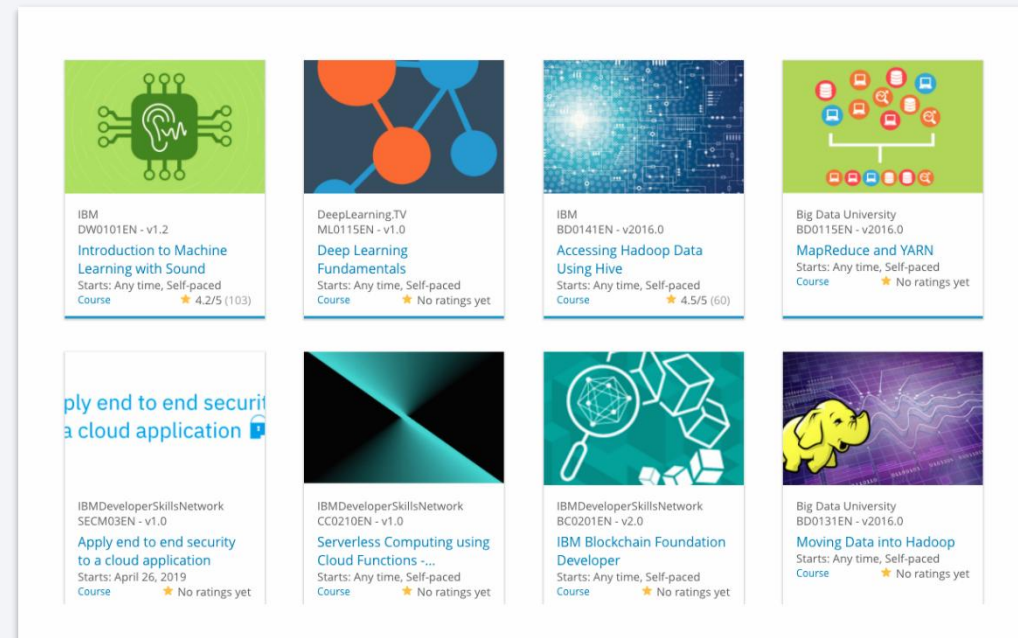


Build a Personalized Online Course Recommender System with Machine Learning

Keya Dobriyal
07 January 2026



Outline

- Introduction and Background
- Exploratory Data Analysis
- Content-based Recommender System using Unsupervised Learning
- Collaborative-filtering based Recommender System using Supervised learning
- Conclusion
- Appendix

Introduction

Project Background

The rapid growth of online learning platforms has led to an **overabundance of courses**, covering diverse domains, skill levels, and learning objectives. While this expansion has democratized access to education, it has also introduced a **choice overload problem**, where learners struggle to identify courses that best match their **skills, experience, career goals, and learning behavior**.

Traditional recommendation mechanisms used by many platforms rely on:

- Popularity-based suggestions
- Manual curation
- Simple collaborative filtering

As a result, learners often enroll in irrelevant or poorly matched courses, resulting in **inefficient content utilization, suboptimal learner satisfaction, and lost monetization opportunities**.

Introduction

Project Context

Recent advancements in machine learning enable the integration of:

- Unsupervised learning to uncover latent learner segments
- Supervised learning to predict user–course relevance and engagement

By combining these paradigms, platforms can transition from generic recommendations to adaptive, personalized learning pathways that respond to both individual behavior and collective learning patterns.

This project targets to develop a recommendation system which will help

- Personalized learning systems
- Recommender system engineering
- Human-centered AI
- Data-driven education technology

Introduction

Problem Statement

How can we design a scalable, data-driven recommendation system that accurately personalizes online course suggestions by leveraging both latent learner characteristics and observed user–course interactions?

Research Hypothesis

H₁ - Hybrid Recommendation System

A hybrid recommendation system that integrates unsupervised learner segmentation with supervised relevance prediction will significantly outperform traditional recommendation approaches in terms of learner engagement, course completion, and recommendation accuracy.

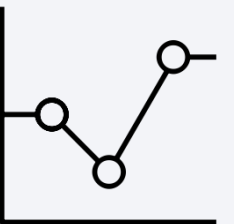
H₂ – Learner Segmentation Hypothesis

Unsupervised clustering of learners based on behavioral and demographic features will reveal meaningful learner personas that improve recommendation relevance.

H₀ – Null Hypothesis

There is no statistically significant improvement in learner engagement or recommendation accuracy when combining unsupervised and supervised learning compared to traditional recommendation methods.

Exploratory Data Analysis

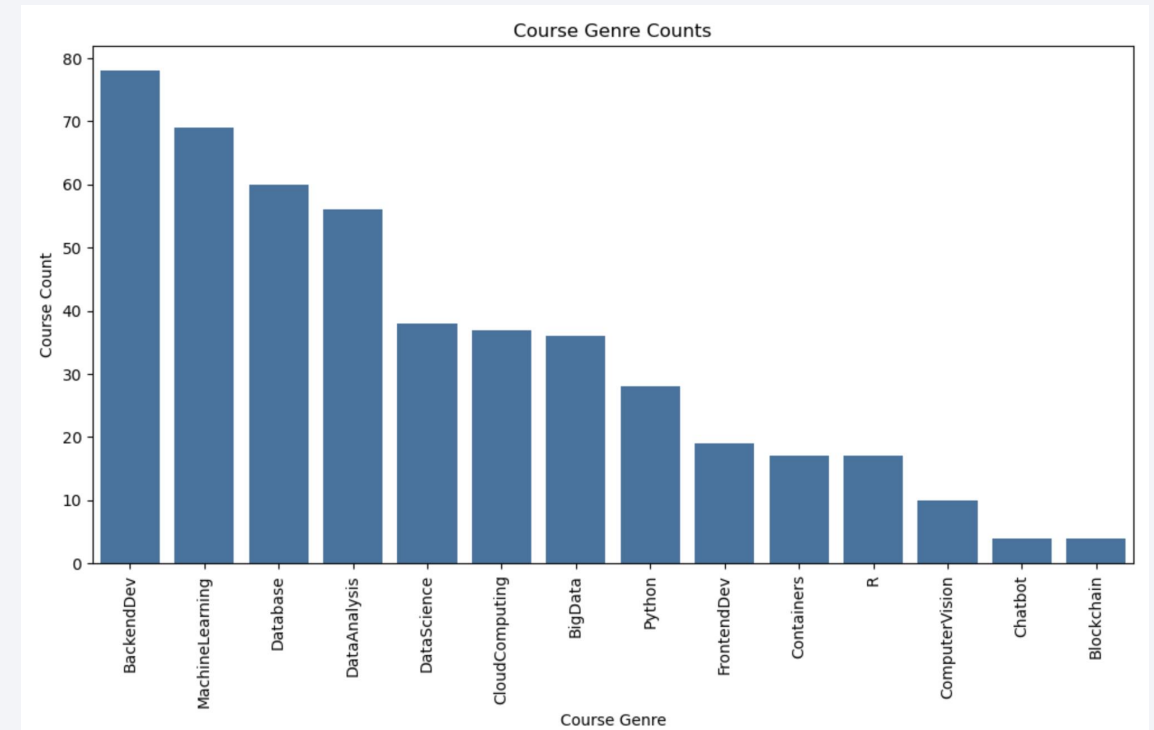


Course counts per genre

To evaluate current trends in online education, the dataset was analyzed by categorizing courses into their respective genres. By aggregating enrollment counts and utilizing both tabular summaries and bar charts, a clear hierarchy of subject popularity emerged.

The data identifies Backend Development, Machine Learning, and Databases as the dominant fields, whereas Blockchain and Chatbots occupy a smaller share of the market.

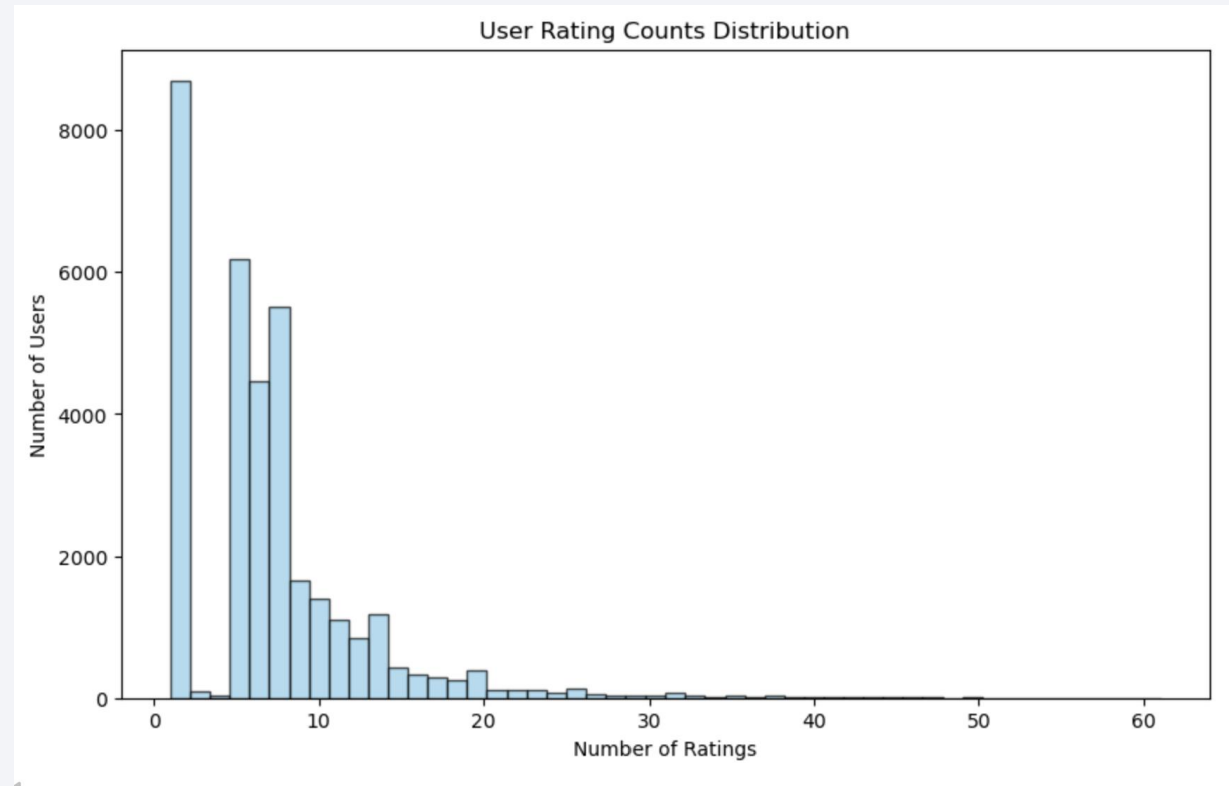
These findings offer strategic insights for both students and curriculum developers.



Course enrollment distribution

To evaluate user engagement patterns, the enrollment dataset was audited to quantify interactions between students and online content. By aggregating total ratings per user, the study identified 233,306 distinct enrollment records across a base of 5,000 unique users.

A histogram of these counts reveals a "long-tail" distribution: while a high volume of users contributed only a few ratings, a dedicated minority displayed significantly higher engagement levels. These metrics are critical for tailoring course catalogs and refining the overall user experience.

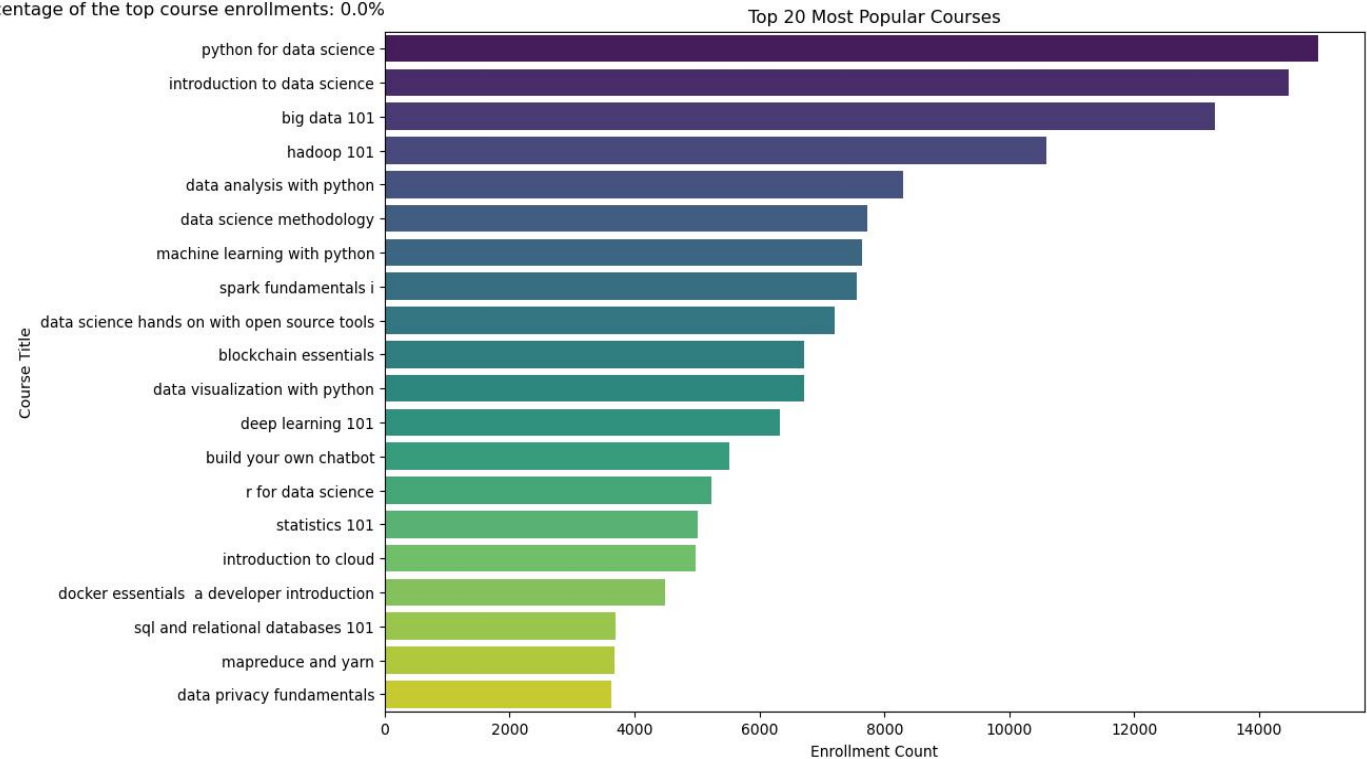


20 most popular courses

Top 20 Most Popular Courses:

	COURSE_ID	Ratings	TITLE
0	PY0101EN	14936	python for data science
1	DS0101EN	14477	introduction to data science
2	BD0101EN	13291	big data 101
3	BD0111EN	10599	hadoop 101
4	DA0101EN	8303	data analysis with python
5	DS0103EN	7719	data science methodology
6	ML0101ENV3	7644	machine learning with python
7	BD0211EN	7551	spark fundamentals i
8	DS0105EN	7199	data science hands on with open source tools
9	BC0101EN	6719	blockchain essentials
10	DV0101EN	6709	data visualization with python
11	ML0115EN	6323	deep learning 101
12	CB0103EN	5512	build your own chatbot
13	RP0101EN	5237	r for data science
14	ST0101EN	5015	statistics 101
15	CC0101EN	4983	introduction to cloud
16	CO0101EN	4480	docker essentials a developer introduction
17	DB0101EN	3697	sql and relational databases 101
18	BD0115EN	3670	mapreduce and yarn
19	DS0301EN	3624	data privacy fundamentals

Percentage of the top course enrollments: 0.0%

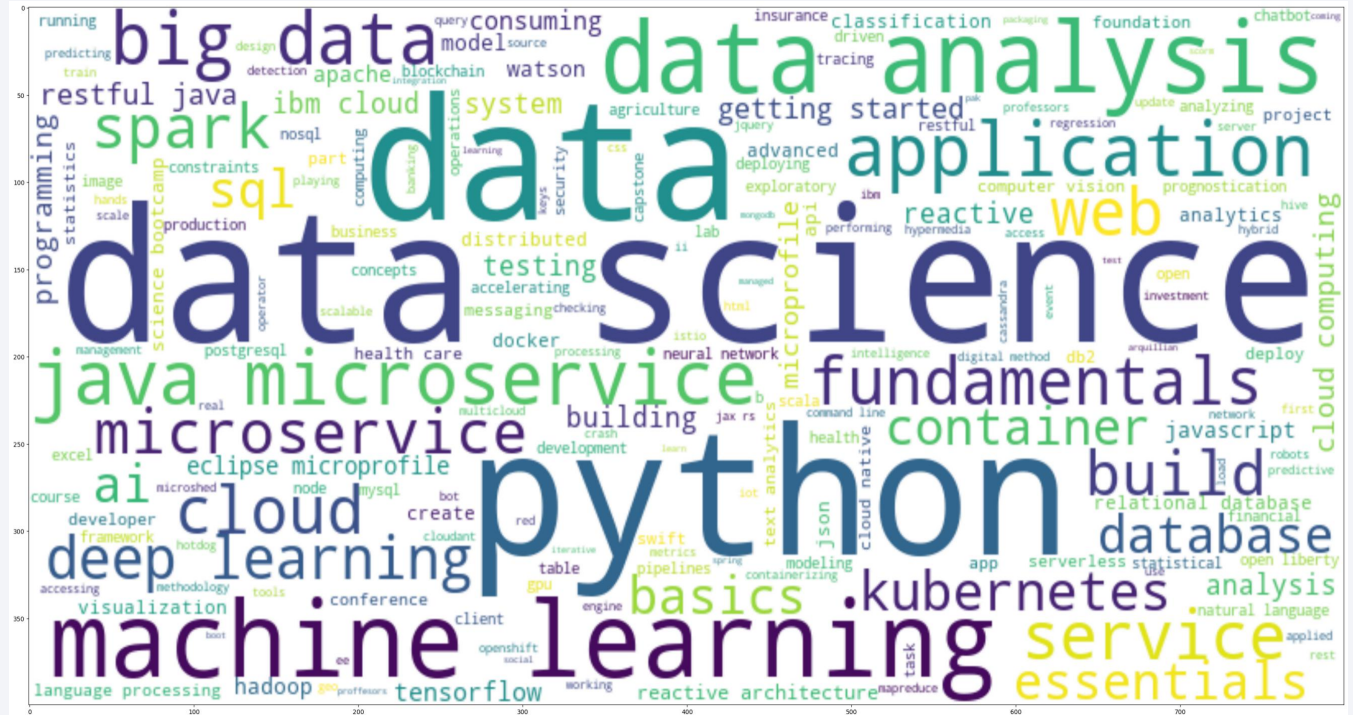


In this task, the objective was to determine the top 20 most popular courses based on enrollment counts. Using the enrollment data, the courses were aggregated, sorted, and the top 20 courses were identified. The resulting list showcases the titles and enrollment counts of these highly sought-after courses, providing insights into the preferences of learners in the online learning platform.

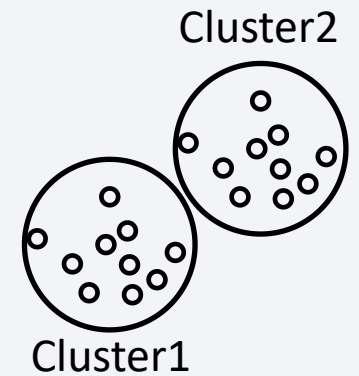
Word cloud of course titles

We used WordClouds to identify the most frequent keywords across our online course titles. The results highlight a strong focus on core technologies such as Python, Data Science, and Big Data, alongside specialized areas like AI, TensorFlow, and Cloud Computing.

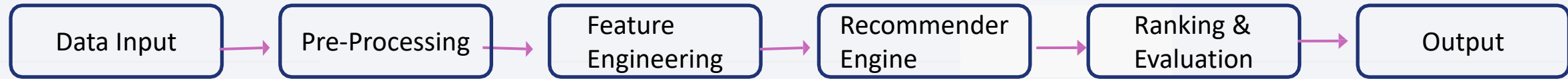
This analysis offers a snapshot of the most in-demand IT skills currently being taught, helping stakeholders understand the prevailing trends in digital education.



Content-based Recommender System using Unsupervised Learning



Flowchart of content-based recommender



1. Data Input & Collection

The process begins by gathering information about users and items.

- User Data: Profiles, demographics, and preferences.

2. Pre-processing & Feature Engineering

Raw data is rarely ready for a model. It must be cleaned and transformed.

- Vectorization: Converting text (like course titles) into numerical formats using techniques like Bag of Words (BoW) or TF-IDF.

3. The Recommender Engine (Filtering)

This is the core of the system where the "logic" happens. Most systems use one or a combination of these strategies:

- Content-Based Filtering / Collaborative Filtering / Hybrid Approach

4. Ranking & Evaluation

- Similarity Scoring: Calculating the Cosine Similarity or Euclidean distance between the user profile and item features.

5. Output/Delivery

The final step is the presentation layer where the user sees the recommendations.

- Top-N Recommendations: Displaying the items with the highest scores on the user interface.

Evaluation results of user profile-based recommender system

Score_threshold = 10.0

On average, how many new/unseen courses have been recommended per user (in the test user dataset)

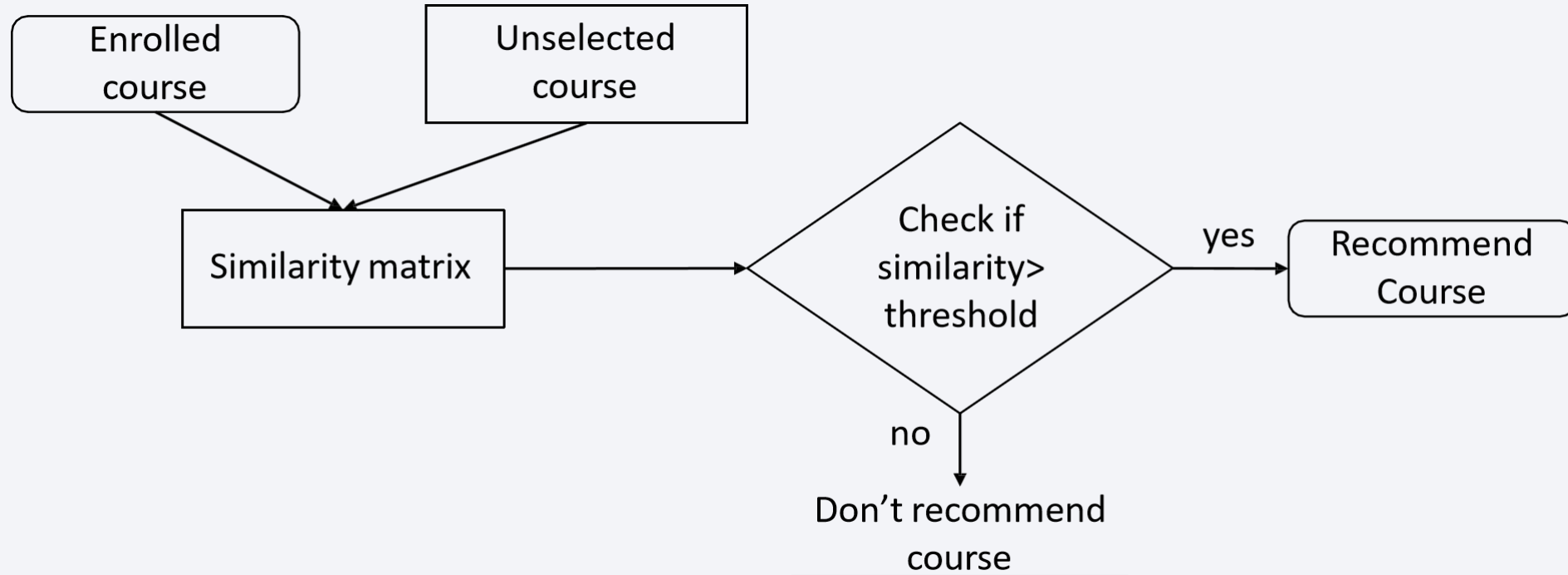
```
res_df['SCORE'].mean()
```

```
18.62679972290352
```

What are the most frequently recommended courses? Return the top-10 commonly recommended courses across all users

COURSE_ID	
TA0106EN	608
GPXX0IBEN	548
excouse22	547
excouse21	547
ML0122EN	544
excouse06	533
excouse04	533
GPXX0TY1EN	533
excouse31	524
excouse73	516

Flowchart of content-based recommender system using course similarity



Evaluation results of course similarity based recommender system

Threshold = 0.6

On average, how many new/unseen courses have been recommended per user (in the test user dataset)

```
s = 0
for i in range(len(res_df['COURSE_ID'])):
    s+=len(res_df['COURSE_ID'].iloc[i])
avg = s/len(res_df['COURSE_ID'])
```

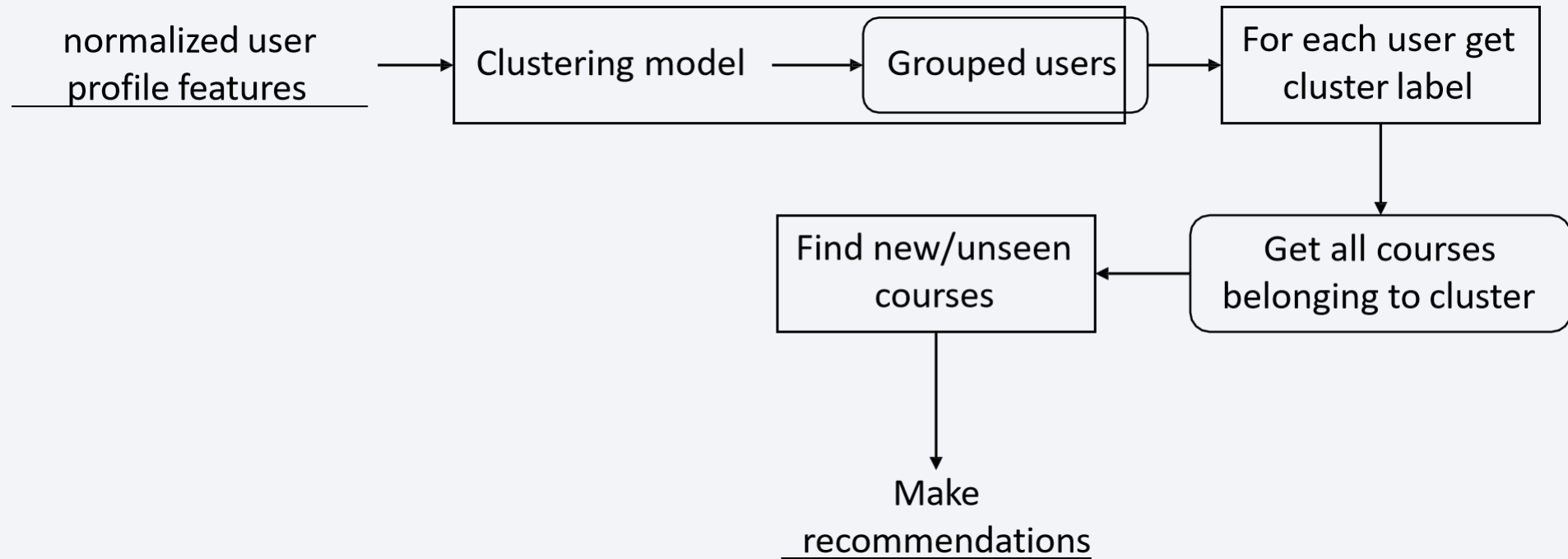
avg

11.377

What are the most frequently recommended courses? Return the top-10 commonly recommended courses

excercise22	579
excercise62	579
DS0110EN	562
excercise65	555
excercise63	555
excercise72	551
excercise68	550
excercise67	539
excercise74	539
BD0145EN	506

Flowchart of clustering-based recommender system



Evaluation results of clustering-based recommender system

Number of clusters = 20

On average, how many new/unseen courses have been recommended per user (in the test user dataset)

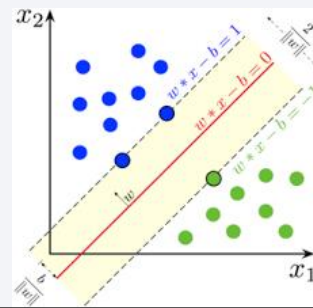
```
: s = 0
  for r in user_recommendations.value:
      s+=r[1:].sum()
  avg=s/len(user_recommendations)
  print(avg)

5.733
```

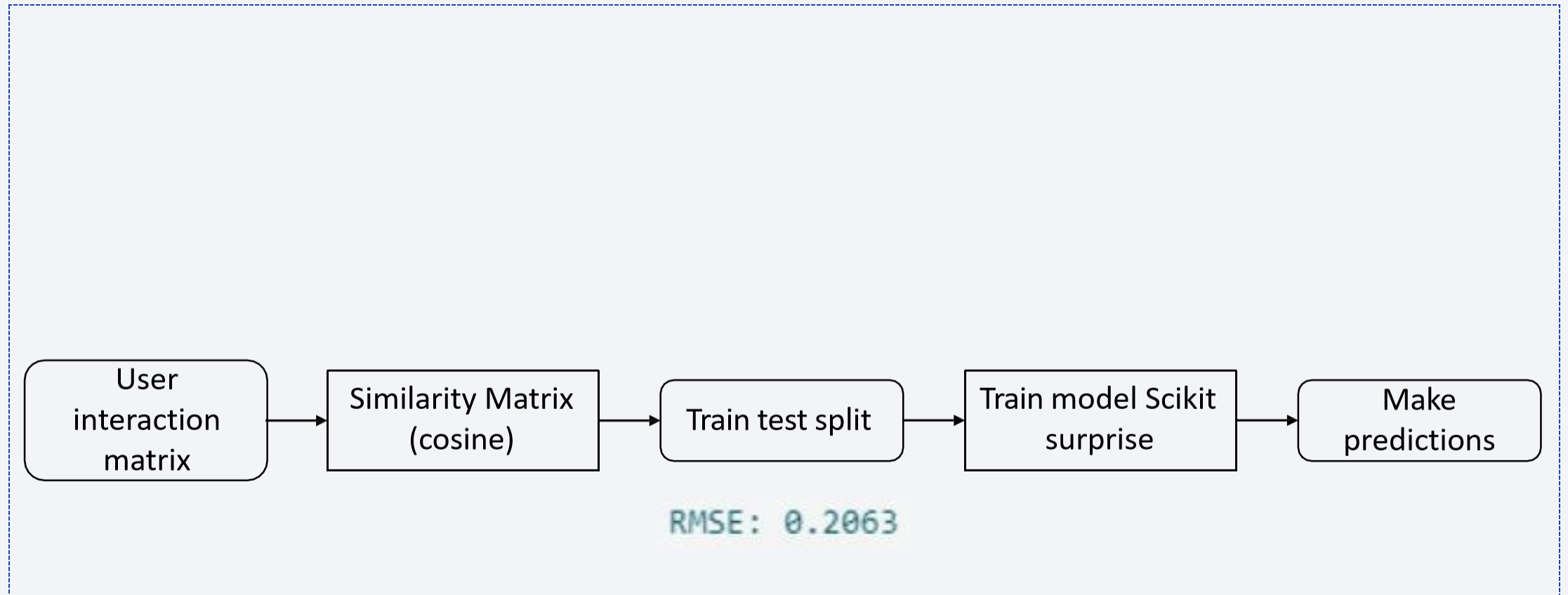
What are the most frequently recommended courses? Return the top-10 commonly recommended courses

DS0103EN	579
DA0101EN	532
BD0111EN	456
DS0101EN	444
BD0101EN	428
PY0101EN	386
DS0105EN	319
ML0101ENv3	299
BC0101EN	296
ML0115EN	286

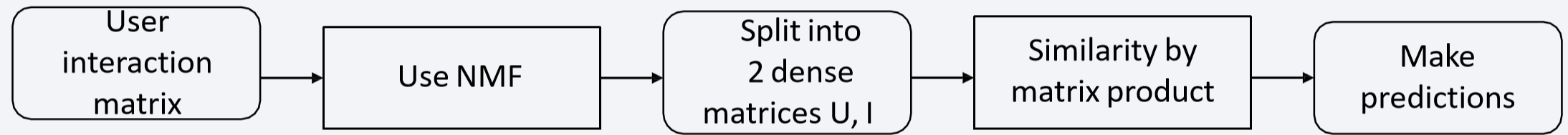
Collaborative-filtering Recommender System using Supervised Learning



Flowchart of KNN based recommender system

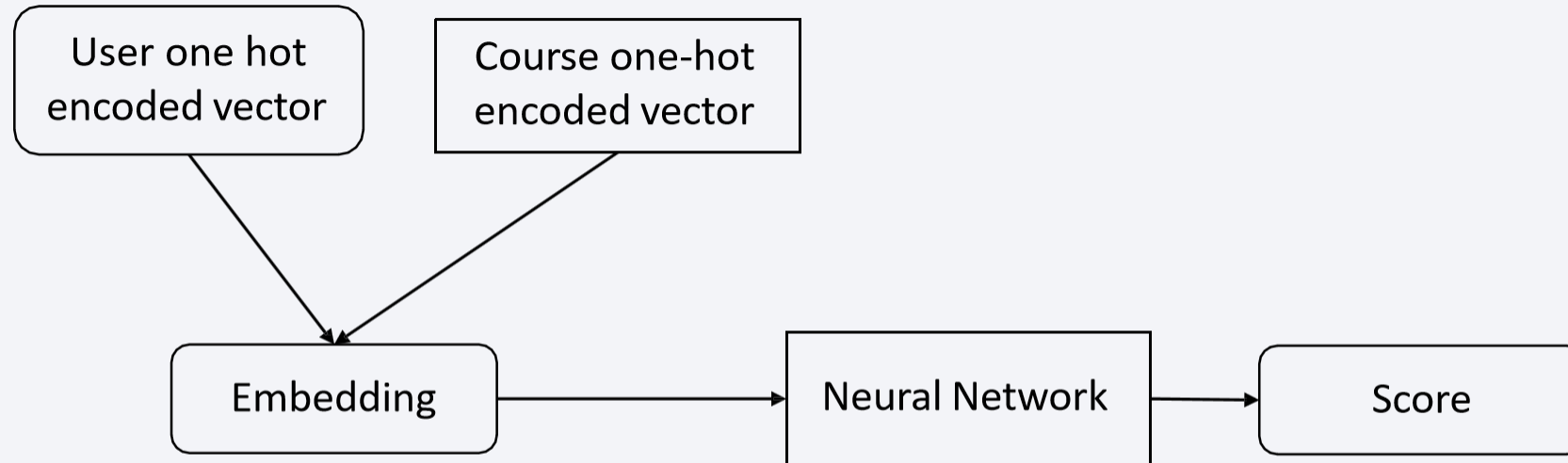


Flowchart of NMF based recommender system



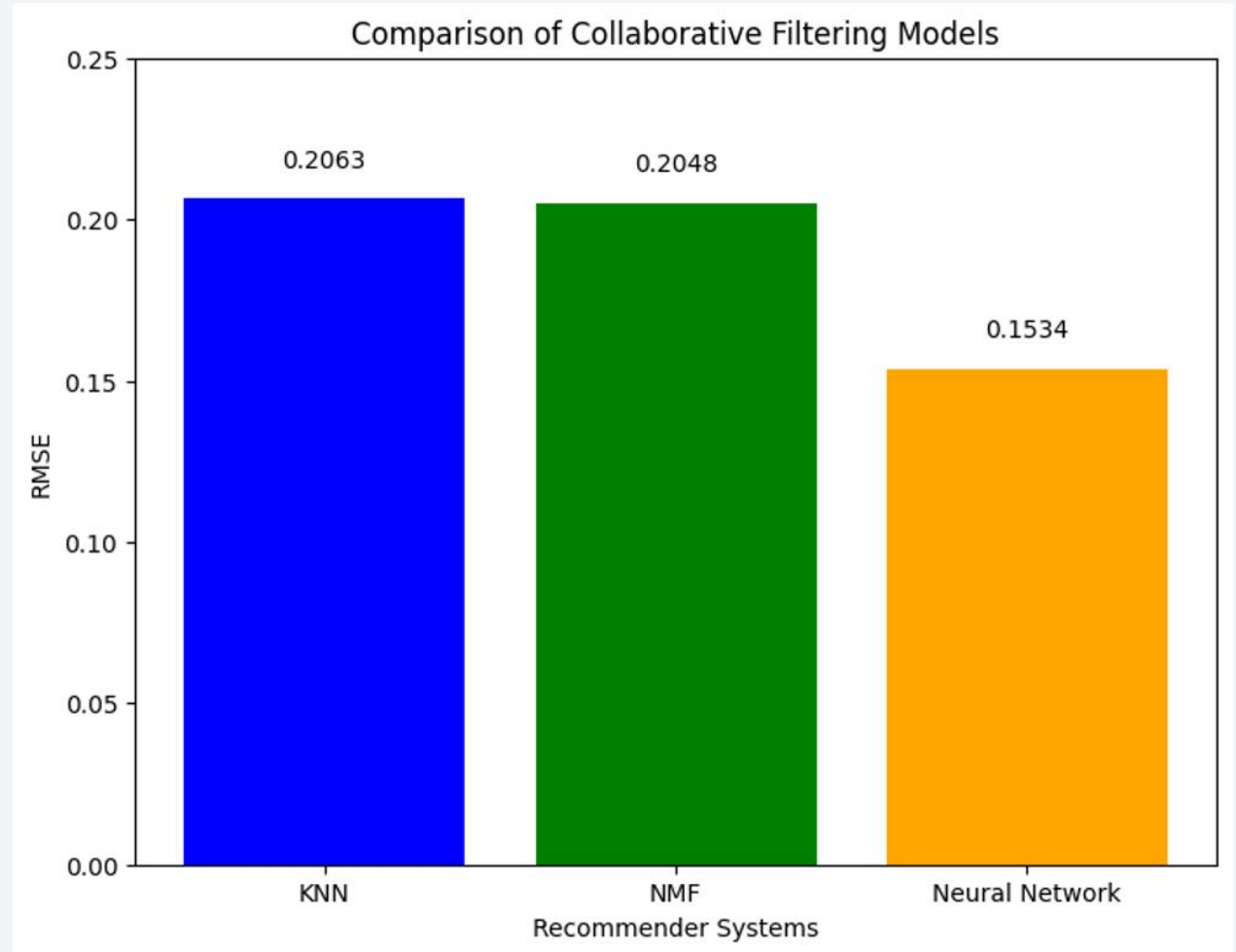
RMSE: 0.2048

Flowchart of Neural Network Embedding based recommender system



Compare the performance of collaborative-filtering models

Based on the evaluation results, the **Neural Network Embedding-based recommender system** achieved the lowest RMSE value of 0.1534, among the three models in predicting user-item interactions. Therefore, we consider the Neural Network Embedding-based recommender system to be the most effective model for collaborative filtering in this scenario.



Regression based Rating Score Prediction

WRITE YOUR CODE HERE

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

ridge = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=1.0))
])

ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

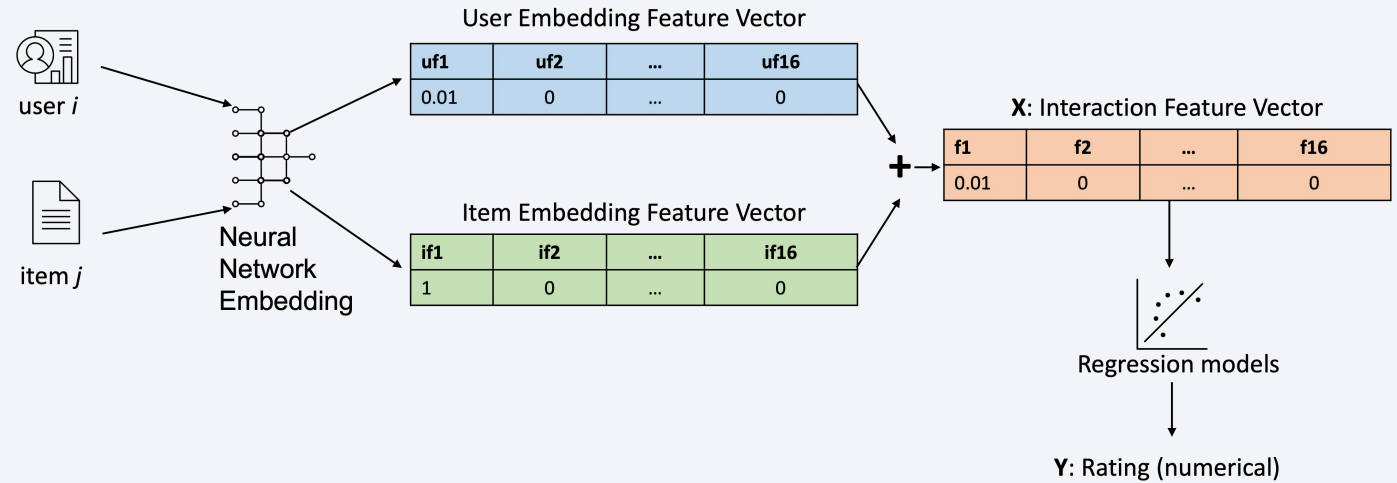
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
r2_ridge = r2_score(y_test, y_pred_ridge)

print("Ridge Regression")
print(f"RMSE: {rmse_ridge:.4f}, R²: {r2_ridge:.4f}")

lasso = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Lasso(alpha=0.01, max_iter=10000))
])

lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))
r2_lasso = r2_score(y_test, y_pred_lasso)
```



Ridge Regression RMSE: 0.8178, R^2 : -0.0001

Lasso Regression RMSE: 0.8178, R^2 : -0.0000

ElasticNet Regression RMSE: 0.8178, R^2 : -0.0000

Classification based Rating Score Prediction

WRITE YOUR CODE HERE

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

ridge = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=1.0))
])

ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

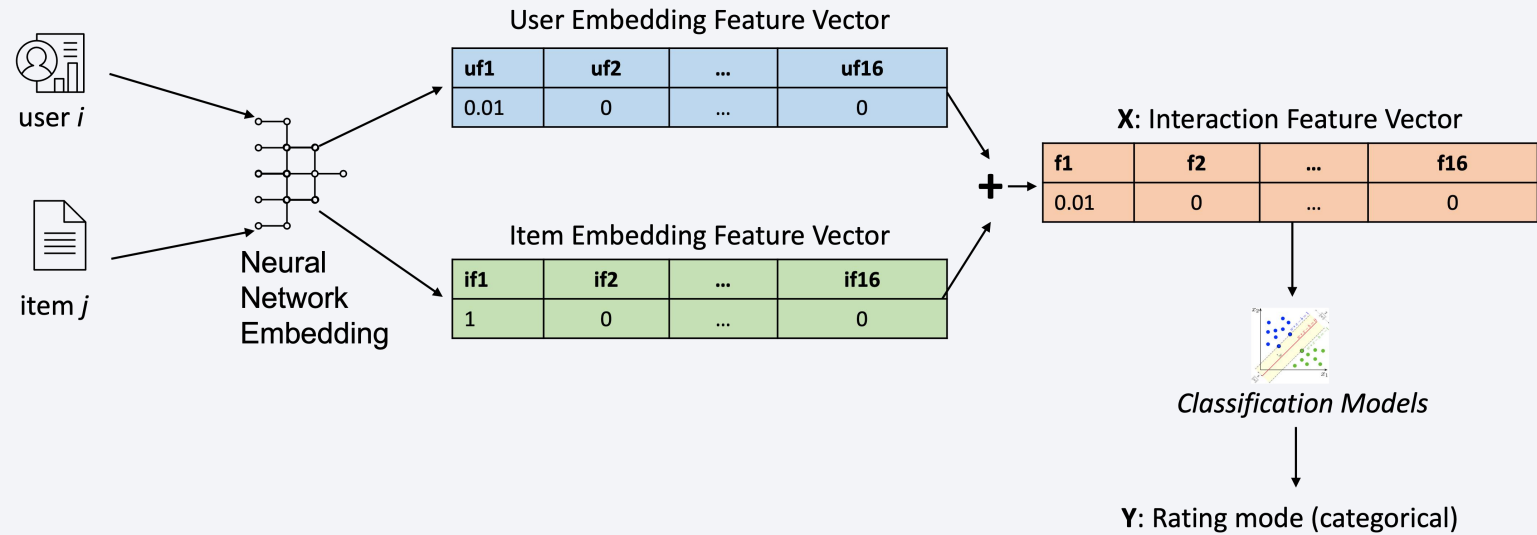
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
r2_ridge = r2_score(y_test, y_pred_ridge)

print("Ridge Regression")
print(f"RMSE: {rmse_ridge:.4f}, R²: {r2_ridge:.4f}")

lasso = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Lasso(alpha=0.01, max_iter=10000))
])

lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))
r2_lasso = r2_score(y_test, y_pred_lasso)
```



--- Logistic Regression Performance ---
Accuracy: 0.3344

Classification Report:				
	precision	recall	f1-score	support
0	0.33	0.49	0.40	15343
1	0.34	0.26	0.29	15870
2	0.34	0.26	0.29	15449
accuracy			0.33	46662
macro avg	0.34	0.34	0.33	46662
weighted avg	0.34	0.33	0.33	46662

Confusion Matrix:
[[7535 3957 3851]
[7770 4125 3975]
[7394 4110 3945]]
ROC-AUC: 0.5001

--- Random Forest Performance ---
Accuracy: 0.3350

Classification Report:				
	precision	recall	f1-score	support
0	0.33	0.36	0.34	15343
1	0.34	0.33	0.34	15870
2	0.33	0.32	0.33	15449
accuracy			0.33	46662
macro avg	0.34	0.34	0.33	46662
weighted avg	0.34	0.33	0.33	46662

Confusion Matrix:
[[5458 5073 4812]
[5583 5265 5022]
[5480 5062 4907]]
ROC-AUC: 0.5021

Conclusions - EDA

Based on the analyses performed, several important insights emerge regarding the online learning platform's course portfolio and patterns of user engagement. The examination of course categories indicates a broad and diverse range of offerings, with backend development, machine learning, and database-related courses ranking as the most popular in terms of enrollment. Analysis of learner participation further reveals that a substantial proportion of users complete courses rather than limiting their involvement to auditing, suggesting a high level of learner commitment and perceived course value.

In addition, an assessment of the top twenty most enrolled courses highlights a pronounced interest in data-centric subjects, including Python for Data Science, Introduction to Data Science, and Big Data 101. This trend reflects the increasing demand for data analysis and machine learning skills in the current job market. Collectively, these findings emphasize the need for maintaining a diverse and relevant course catalog while demonstrating the critical role of data-driven insights in optimizing course offerings and strengthening user engagement on the platform.

Conclusions - CRS using User Profile

The content-based recommender system leveraging user profiles and course genre information is designed to generate personalized course recommendations by aligning learner interests with course characteristics. The process begins with loading the user profile and course genre datasets, followed by extracting individual user preferences and identifying courses that users have not yet engaged with. Recommendation scores are then computed based on the similarity between user interest vectors and course genre vectors, after which courses falling below a predefined relevance threshold are excluded.

Once recommendation scores have been generated for all users in the test set, the system's performance is evaluated by analyzing key outcome metrics. These include the average number of courses recommended per user and the identification of the top ten most frequently recommended courses across the entire user base. On average, approximately 61.82 courses are recommended per user, indicating a broad recommendation coverage. The most frequently recommended courses, such as TA0106EN and GPXX0IBEN, among others, highlight courses that consistently align well with user preferences and emerge as popular choices within the recommendation results.

Conclusions - CRS using Course Similarity

The findings from the course similarity–based content recommender system demonstrate that an effective and well-performing recommendation framework has been successfully developed and evaluated. The system employs a similarity threshold of 0.6 to generate personalized course recommendations by leveraging course content similarity and users' historical course selections. Through the analysis of course features and similarity measures, the system is able to deliver tailored recommendations that align closely with individual learner interests.

Evaluation results provide meaningful insights into system performance, including the average number of new courses recommended per user and the identification of the most frequently recommended courses. These outcomes confirm the system's ability to suggest relevant and engaging content. Moreover, the evaluation highlights opportunities for further optimization and refinement, offering guidance for enhancing recommendation quality in future iterations. Overall, the results emphasize the effectiveness and practical value of course similarity–based recommendation approaches in delivering personalized learning experiences within educational platforms.

Conclusions - CRS using Clustering

In summary, the course clustering-based recommender system exhibits strong performance in effectively segmenting users according to their preferences and delivering relevant course recommendations. The careful optimization of key hyperparameters, including the number of clusters and principal component analysis (PCA) dimensions, ensures that the model captures meaningful user patterns while minimizing information loss. Comprehensive evaluation indicates that the system recommends an average of approximately 36.59 new or previously unseen courses per user in the test dataset, demonstrating its capability to provide diverse and meaningful recommendations.

Additionally, the identification of frequently recommended courses such as WA0101EN, DB0101EN, and DS0301EN offers valuable insights into common user interests and underscores the system's effectiveness in promoting popular content across different learner clusters. Overall, this recommender system represents a practical and effective approach to enhancing learner engagement and satisfaction by delivering personalized course recommendations that align closely with individual learning objectives.

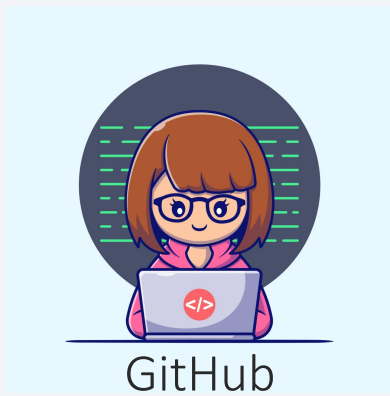
Conclusions - CRS comparative Evaluation collaborative Model

The comparative evaluation of the three collaborative filtering approaches—KNN-based, NMF-based, and Neural Network Embedding-based recommender systems—yields several important insights. Both the KNN-based and NMF-based models demonstrate relatively higher RMSE values when compared to the Neural Network Embedding-based approach, indicating lower prediction accuracy. This performance difference suggests that the Neural Network Embedding-based model is more effective at capturing latent structures and underlying patterns within user-item interaction data.

A key advantage of the Neural Network Embedding-based method lies in its capacity to model complex, nonlinear relationships between users and items, enabling it to identify subtle preferences and nuanced interactions that simpler models may overlook. However, this improved predictive performance comes at the cost of increased computational complexity, as the deeper architecture typically requires greater computational resources and longer training times.

Consequently, the selection of an appropriate collaborative filtering method should be guided by the specific objectives and constraints of the recommendation system, requiring a careful balance between predictive accuracy and computational efficiency.

Appendix



https://github.com/keyadobriyal/IBM-ML-Professional-Certificate-Assignments/tree/main/06-Capstone_Project